POT JS
FUNCTIONS

1.0.2

POT JS function descriptions with examples.
For an introduction and tutorial, see the manual.

ASYNCHRONOUS FUNCTIONS

The core functions – `put()`, `get()`, `delete()`, `save()`, `load()` – return a promise. If an error is encountered, the promise is rejected, and the error is thrown. All functions whose names do not end in `Sync` are asynchronous. Their use is preferred.

SYNCHRONOUS FUNCTIONS

There are synchronous variants for most functions, that add `Sync` to the function name: `putSync()`, `getSync()` etc. They can make sense for fast prototyping and they can be used in synchronous functions.

But synchronous functions do not work in the browser when connecting to a network, as opposed to working with an in-memory POT. That connection is determined by using `new()` with or without its optional parameters. (`new(null, null)` also effects that in-memory storage is used.)

Synchronous calls never throw errors, they are returning an Error object instead in case of a failure.

TYPE-AWARE FUNCTIONS

`get()` returns the type that was stored using `put()`. The type information is persisted with the value. A value can have the type `boolean`, `number`, `string`, `Uint8Array`, or `null`.

RAW FUNCTIONS

The standard functions – `put()` and `get()` – operate with type information that is being written and read as first byte of the raw byte value that is actually being stored.

Functions with `Raw` in their name store and return the bytes as they are, ignoring any type information. They exist to design structures or attach meta data to values that is more complex than just the primitive type. Certain functions – `getBoolean()`, `getString()`, `getNumber()` – read a raw byte value assuming that it will be a boolean, string, or number respectively, also assuming that the raw bytes do not include a type code byte.

Standard and raw calls should not be mixed to avoid data corruption. A project should decide for one or the other to err on the safe side. Standard typed-coded are `put()`, `putSync()`, `get()`, `getSync()`. Raw are `putRaw()`, `putRawSync()`, `getRaw()`, `getRawSync()`, `getBoolean()`, `getBooleanSync()`, `getString()`, `getStringSync()`, `getNumber()`, `getNumberSync()`.

CREATE AND SAVE KVS

NEW KVS

pot.new([<bee url>, <batch id>]) → promise of KVS

pot.newSync([<bee url>, <batch id>]) → KVS or Error

<bee url> : http address of the API of a Bee Swarm client.
<batch id> : 64-digit hex string of a Swarm postage batch.

Create a new KVS object.

The parameters are optional. Leaving them out creates a key-value store in-memory. Giving both, connects to a Swarm Bee node to store the KVS there (with `save()`). `New(null, null)` also selects in-memory storage.

Although a promise is returned, this function does not accept a timeout as it is independent of external events and resolves, immediately.

In-memory:

```
(async () => {

      await pot.ready()
      kvs = await pot.new()
      ...
})()
```

Connecting to a Swarm network. In the example, a local network:

```
(async () => {

      await pot.ready()
      const bee_url = "http://localhost:1633"
      const batch_id =
      "6792a183adafdac3a0a1a1e65b435406aff8f4343f980b16cabafd4a8525c0aa"
      kvs = await pot.new(bee_url, batch_id)
      ...
})()
```

Synchronous use:

```
globalThis.onPotInitialized = () => {

      kvs = pot.newSync(bee_url, batch_id)
      ...
}
```

LOAD NEW KVS FROM SAVE REFERENCE

Create a new Swarm KVS from a save reference.

```
pot.load(<reference>[, <bee url>, <batch id>[, <timeout>]])
      → promise of KVS

pot.loadSync(<reference>[, <bee url>, <batch id>[, <timeout>]])
      → KVS or Error

<reference>  : 32-digit hex string obtained from save().
<bee url>    : http address of the API of a Bee Swarm client.
<batch id>   : 64-digit hex string of a Swarm postage batch.
<timeout>    : milliseconds.
```

The bee url and batch id parameters have to match (obviously) where the KVS was stored-to that is to be retrieved. That is, what was set in the `new()` call originally. There is no direct way to migrate KVSs between different storages.

The optional timeout is in milliseconds. The loading will be canceled if loading is not complete before that timeout. For the async call, the promise will reject, throwing an error, the sync call will return an error. `bee_url` and `batch_id` can be given as `null` to give the timeout as fourth argument. Like with all functions that have a timeout, if no timeout is given, `load()` for its part never times out but other underlying mechanisms may at some point still timeout, throwing an error or dying silently.

```
ref = await kvs.save()

kvs2 = await pot.load(ref)

kvs3 = await pot.load(ref, null, null, 3000) // 3 sec timeout
      .catch(...)

kvs = pot.newSync()

kvs.putSync(key1, val1)

ref = kvs.saveSync()

kvs2 = pot.loadSync(ref)
```

SAVE

```
kvs.save([<timeout>]) → promise of reference

kvs.saveSync([<timeout>]) → reference or Error

<timeout>    : milliseconds.
<reference>  : 32-digit hex string.
```

Actually storing a KVS to the network. Put actions always operate in-memory first, until the entire map is saved to the network. Save does not accept a `bee_url` or `batch_id` argument. It uses what was determined when the KVS was created with `new()` or `load()`. This function is not meaningful for in-memory settings but it exists also for KVS created in-memory to ease development and testing.

For a discussion of the save reference, see the manual, chapter Instructions.

The optional timeout is in milliseconds. The saving will be canceled if it is not complete before that time. For the async call, the promise will reject, throwing an error, the sync call will return an error.

```
await kvs.put(key1, val1)
ref = await kvs.save(100)     // one tenth of a second as timeout
kvs.putSync(key2, val2)
ref2 = kvs.saveSync()
```

STORING, RETRIEVING, DELETING KEY-VALUE PAIRS

The asynchronous, type-aware functions – `get()` and `put()` – are the preferred ones for production use, unless you are experimenting or really need the other ones.

PUT

Store a value, asynchronously, and type-aware.

After storing with `put()`, `get()` automatically converts the stored value back to the right type. (`getRaw()` would return all bytes – including the first byte that stands for the type – as `Uint8Array`.)

`kvs.put(<key>, <value>[, <timeout>]) → promise of null`

`kvs.putSync(<key>, <value>[, <timeout>]) → null or Error`

```
<key>       : string, number, or Uint8Array. 32 bytes max.
<value>     : string, number, Uint8Array, Boolean, or null.
<timeout>   : milliseconds.
```

Keys are zero-padded to 32 bytes. Values can be up to 100,000 bytes or more. See the manual, chapter Instructions, for discussions of key ambiguity and value size.

The optional timeout is in milliseconds. The `put` will be canceled if it is not complete before that time. For the async call, the promise will reject, throwing an error, the sync call will return an error. If no timeout is given, `put()` never times out.

```
kvs = await pot.new()
try {
    await kvs.put("fox", "The quick brown fox jumps!")
} catch(e) {
    ...
}
```

GET

kvs.get (<key>[, <timeout>]) ⟶ promise of value

kvs.getSync(<key>[, <timeout>]) ⟶ value or Error

<key> : string, number, or Uint8Array. 32 bytes max.
<value> : string, number, Uint8Array, Boolean, or null.
<timeout> : milliseconds.

Retrieve a value from the KVS, asynchronously and type-aware. (The first of the stored bytes is interpreted as type information, as written with `put()`, but not `putRaw()`).

The optional timeout is in milliseconds. The `get` will be canceled if the `get` is not complete before that timeout. For the async call, the promise will reject, throwing an error, the sync call will return an error. If no timeout is given, `get()` never times out.

```
try {
        value = await kvs.get(key)
} catch(e) {
        ...
}

result = kvs.getSync(key)
if(result instanceof Error) {
        ...
}
```

DELETE

kvs.delete (<key>[, <timeout>]) ⟶ promise of null

kvs.deleteSync(<key>[, <timeout>]) ⟶ null or Error

<key> : string, number, or Uint8Array. 32 bytes max.
<timeout> : milliseconds.

Drop a key-value pair from the key-value store.

The optional timeout is in milliseconds. The deletion will be canceled if it is not complete before that timeout. For the async call, the promise will reject, throwing an error, the sync call will return an error. If no timeout is given, `delete()` never times out.

```
try {
      await kvs.delete(key)
} catch(e) {
      ...
}


result = kvs.deleteSync(key)
if(result instanceof Error) {
      ...
}
```

ON INITIALIZATION

`pot.ready() → Promise of null`

Use with `await` to wait until the pot object is initialized. Per design, the WASM executable has to be started and complete its main routine. This is triggered by `pot-node.js` or `pot-web.js`. Once this is done, `pot.ready()` resolves and the `pot.*` methods can be used from that point on.

```
await pot.ready()
kvs = await pot.new()

...
```

`globalThis.onPotInitialized() → none`

This function is called from the WASM executable when the initialization of the `pot` object is done. It can be used as a synchronous substitute for `pot.ready()`. But it can also be defined asynchronously to allow for `await` in its body. If you don't define this function there is no harm.

```
globalThis.onPotInitialized = () => {
      kvs = pot.newSync()
      kvs.putSync("hello", "POT!")
      ...
}
```

RAW BYTE ACCESS

To store `Uint8Arrays` directly – e.g., to attach custom meta data beyond type information, or to inspect the byte level beneath the type-encoding as-is – values can be stored and retrieved as 'raw' bytes without the leading, type encoding byte that `put()`, `get()`, `putSync()`, and `getSync()` use to identify types. The raw mode is thus not compatible with the standard, type-aware access and raw values should not be retrieved with `get()` or `getSync()`.

Instead, these values can be retrieved raw, by `getRaw()` and then casted, or by having them converted immediately to a desired type with `getBoolean()`, `getNumber()`, and `getString()`.

Retrieving values with `get()` or `getSync()` that were stored with `putRaw()` leads to errors when the first byte gets interpreted as type information and then discarded. The same misinterpretation arises vice-versa, when a value stored with `put()` or `putSync()` is retrieved with `getBoolean()`, `getString()` or `getNumber()`. In these cases, the type code byte would be mixed into the value and a wrong value be returned. You might decide to use only one set of functions or the other in any given project to safely duck a nasty class of errors.

STORE RAW BYTES

```
kvs.putRaw(<key>, <value> : Uint8Array[, <timeout>])
     → promise of null

kvs.putRawSync(<key>, <value> : Uint8Array[, <timeout>])
     → null or Error

<key>       : string, number, or Uint8Array. 32 bytes max.
<value>     : Uint8Array.
<timeout>   : milliseconds.
```

Store a raw `Uint8Array byte` array, without type information.

The optional timeout is in milliseconds. The put will be canceled if it is not complete before that timeout. For the async call, the promise will reject, throwing an error, the sync call will return an error. If no timeout is given, `putRaw()` never times out.

RETRIEVE FROM RAW BYTES

```
kvs.getRaw(<key>[, <timeout>]) → promise of Uint8Array

kvs.getRawSync(<key>[, <timeout>]) → Uint8Array or Error

<key>       : string, number, or Uint8Array. 32 bytes max.
<value>     : Uint8Array.
<timeout>   : milliseconds.
```

Retrieve a value as raw `Uint8Bytes` array. Not expecting a type-byte. You can retrieve any value like this, but values stored with `put()` will have as first byte the type-information and the value itself starts with the second byte.

The optional timeout is in milliseconds. The operation will be canceled if it is not complete before the timeout. For the async call, the promise will reject, throwing an error, the sync call will return an error. If no timeout is given, the method will not timeout.

```
kvs.getBoolean(<key>[, <timeout>]) → promise of boolean

kvs.getBooleanSync(<key>[, <timeout>]) → boolean or Error

<key>       : string, number, or Uint8Array. 32 bytes max.
<value>     : boolean.
<timeout>   : milliseconds.
```

Retrieve a value assumed to be stored raw, as a Boolean value. If the first byte is 0, `false` is returned, in all other cases, `true`.

The optional timeout is in milliseconds. The operation will be canceled if it is not complete before the timeout. For the async call, the promise will reject, throwing an error, the sync call will return an error. If no timeout is given, the method will not timeout.

Use `get()` to read a Boolean back that you stored with `put()`, it will automatically come back as `boolean`. Use `getBolean()` to read a Boolean back that was stored with `putRaw()`.

`kvs.getNumber(<key>[, <timeout>]) → promise of number`

`kvs.getNumberSync(<key>[, <timeout>]) → number or Error`

```
<key>       : string, number, or Uint8Array. 32 bytes max.
<value>     : number.
<timeout>   : milliseconds.
```

Retrieve the raw bytes as floating-point number. The value has to be 8 bytes long.

The optional timeout is in milliseconds. The operation will be canceled if it is not complete before the timeout. For the async call, the promise will reject, throwing an error, the sync call will return an error. If no timeout is given, the method will not timeout.

Use `get()` to read a number back that you stored with `put()`, it will automatically come back as number. Use `getNumber()` to read a number back that was stored with `putRaw()`.

`kvs.getString(<key>[, <timeout>]) → promise of string`

`kvs.getStringSync(<key>[, <timeout>]) → string or Error`

```
<key>       : string, number, or Uint8Array. 32 bytes max.
<value>     : string.
<timeout>   : milliseconds.
```

Retrieve raw bytes as string.

The optional timeout is in milliseconds. The operation will be canceled if it is not complete before the timeout. For the async call, the promise will reject, throwing an error, the sync call will return an error. If no timeout is given, the method will not timeout.

Use `get()` to read a string back that you stored with `put()`, it will automatically come back as string. Use `getString()` to read a string back that was stored with `putRaw()`.

SUPPORT FUNCTIONS

HELLO

`pot.hello()`

Write `hello` to browser console or terminal to verify that the WASM executable is up and running.

LOG

`pot.log(message : string)`

Verbosity-sensitive logging.

This functions also serves to verify language-lands roundtrip. If the log message arrives in the console or terminal, the WASM connection from Javascript works.

```
pot.log("hello!")
```

`pot.log()` is subject to the verbosity setting. The default log level that a call to this `pot.log()` function uses is `CRITICAL`. This means that a call to `pot.log()` will always show up, unless verbosity of the system is set to `NONE`. A different log level can be passed to `pot.log()` as optional second parameter.

```
pot.log("hello.", pot.INFO)
```

This example means, that this message will appear in the console or terminal, unless the verbosity setting is `NONE`, `CRITICAL`, or `ERROR`.

For the verbosity log levels see `setVerbosity()`.

VALUE SIZE LIMIT

`pot.setValueSizeLimit(limit : int)`

Set the limit beyond which a value is rejected with an error. This function sets an arbitrary value to protect an application. The limit is in bytes (not characters).

The factual value size limit depends on the underlying system (OS, hardware) setup. For network use with node.js it is beyond 100,000 byte; for in-browser use, with network, it is beyond 10,000,000.

OPTIMIZATION

To increase robustness where needed, optimization allows to switch off resource releases to achieve a potentially more robust operation at the cost of minor resource leakage. For a discussion of the dimension of the leakage, see the manual.

Like all optimization settings, this option exists in case optimization is failing or might be causing errors. In a perfect world it would just be always on.

The default setting is to release resources, the STANDARD level.

`0`	`NONE`	resources are not released, the system leaks slightly, might be more robust.
`1`	`STANDARD`	resources are released, this is the expectable and default behavior.

There are two ways to set optimization. It can be changed anytime with:

`pot.setOptimization(<0 or 1>)`

```
(async () => {

      await pot.ready()
      pot.setOptimization(pot.NONE) // or 0
      map = await pot.new()

})()
```

`globalThis.potjs_Optimization = <0 or 1>`

To set Optimization before the WASM executable is initialized, to suppress even the first two lines it would otherwise log, use `potjs_Optimization`. This works only when it is set before there WASM executable is started. Setting it to a new value after that will not have an effect.

```
var go = new Go()

global.potjs_Optimization = 0 // `pot.NONE` does not exist yet

WebAssembly.instantiate(
      fs.readFileSync("lib/pot.wasm"), go.importObject)
        .then((r) => { go.run(r.instance) })

onPotInitialized = async () => {
      ...
```

VERBOSITY

Verbosity controls the level of detail of the logs written to screen or browser console.

The default setting is `DEBUG`-level verbosity, which results into a high amount of log entries to track a lot of details. For production, `INFO` or `ERROR` will make sense.

There is no general need to allow even errors to make a log entry, as they are also thrown or returned as value by any function call that is affected. `NONE` can therefore be a valid choice.

Levels

Verbosity can be set to these levels:

`0`	`pot.NONE`	no logging, not even errors.
`1`	`pot.CRITICAL`	logs only errors that appear to arise from a POT JS malfunction.
`2`	`pot.ERROR`	programming and runtime errors are also logged.
`3`	`pot.INFO`	general runtime information is logged.
`4`	`pot.DEBUG`	specific data, like put and get keys and values are logged.
`5`	`pot.TRACE`	some execution paths or logged and hex values are not abbreviated.

There are four ways to set the level that are useful in different situations. The examples in `examples/` and the Examples chapter in the manual illustrate the uses.

In general, verbosity can be changed anytime with:

`pot.setVerbosity(level : int)`

```
(async () => {
	await pot.ready()
	pot.setVerbosity(pot.INFO) // or 3
	map = await pot.new()
})()
```

All of the following ways to set verbosity cater to the potential wish to suppress also the first couple of lines that POT JS logs during initialization of the WASM executable.

`globalThis.potjs_verbosity = level : int`

In the special case that `pot-node.js` or `pot-web.js` are not used, to set verbosity before the WASM executable is initialized, to suppress even the first lines it would otherwise log, use `globalThis.potjs_verbosity`. Of course, this works only when it is set before the WASM executable is started. Setting it to a new value after that will not have an effect.

```
var go = new Go()
globalThis.potjs_verbosity = 3 // can't be pot.INFO, it does not exist yet
WebAssembly.instantiate(
	fs.readFileSync("lib/pot.wasm"), go.importObject)
	  .then((r) => { go.run(r.instance) })

onPotInitialized = async () => {
...
```

`<script ... verbosity=level>`

The verbosity level can be set in the `verbosity` attribute of the `script` tag that includes `pot-web.js`. Like with all ways to set verbosity before pot is initialized, it cannot use the pot.* constants but must be a literal.

```
<script src="lib/pot-web.js" wasm="lib/pot.wasm" verbosity="3"></script>
```

`require(module)([wasm path, ] level)`

The verbosity level can also be set in a second parameter group to require, in a separate bracket, as first or second parameter. It cannot use the pot.* constants but must be a literal. In these examples it is in each case the 3:

```
require("pot-node")(3)

require("pot-node")("lib/pot.wasm", 3)
```

DEVELOPMENT

The following functions have no role in production use of POT JS. They are for testing POT JS.

GENERAL TESTING

These functions exist for black box testing. See `examples/suites.js` for examples of their use.

RANDOM KEY

`pot.randKey() ⟶ Uint8Array(32)`

Generate 32 random bytes for key testing.

RANDOM VALUE

`pot.randValue() ⟶ Uint8Array(79–101)`

79 to 101 random bytes for value testing (range taken from Go POT).

RANDOM BUFFER

`pot.randBuffer(n) ⟶ Uint8Array(n)`

Generate n random bytes for value testing.

TYPE-ENCODED BYTES

`pot.type_encoded_bytes(<value>) ⟶ Uint8Array`

Javascript type detection and value encoding with correct leading type byte.

`value types: string, number, Boolean, Uint8Array, null.`

TYPE-DECODED VALUE

`pot.type_decoded_value(encoded bytes : Uint8Array) ⟶ value`

Javascript type detection by leading type byte and according conversion of raw KVS value.

`value types: string, number, Boolean, Uint8Array, null.`

SIMULATION TESTING

The following functions are effective only when the WASM executable has been compiled for the API-side simulation for extended testing. They exist but are all no-ops in the normal production build.

FAULTY PROMISES

`pot.hangingPromise() ⟶ promise of null`

Blocking promise (programmed in Go) with 1 second time out for exception testing.

This function exists but is a no-op in the normal production build.

`pot.panickingPromise() ⟶ rejected promise`

This promise (programmed in Go) immediately panics, i.e., simulates a Go exception.

This function exists but is a no-op in the normal production build.

SET FAULT

`pot.setFault(bool)`

Make the next function call to the mock storage function return an (JS) error to JS.

This function exists but is a no-op in the normal production build.

SET PANIC

`pot.setPanic(bool)`

Make the next function call to the mock storage function panic (on the Go side).

This function exists but is a no-op in the normal production build.

SET DELAY

`pot.setDelay(msec : int)`

Make the next call to a mock storage function stall for msec milliseconds.

This function exists but is a no-op in the normal production build.

SET HANG

`pot.setHang(bool)`

Make the next call to a mock storage function stall indefinitely.

This function exists but is a no-op in the normal production build.